

Documentation Work in Agile Teams: The Role of Documentation Formalism in Achieving a Sustainable Practice

Christoph Johann Stettina¹, Werner Heijstek¹ and Tor Erlend Fægri²

¹LIACS, Leiden University, Niels Bohrweg 1, 2333 CA Leiden, Netherlands
{stettina,heijstek}@liacs.nl

²SINTEF ICT, SP Andersens vei 15b, 7465 Trondheim, Norway
tor.e.fegri@sintef.no

Abstract—As its second guiding principle, agile software development promotes working software over comprehensive documentation. In this paper we investigate alignment between two different documentation practices and agile development. We report upon an experiment conducted to explore the impact of formalism and media type on various dimensions of documentation practice in agile teams. 28 students in 8 teams were divided into two groups: SAD and UML. Group SAD was to update and deliver their high-level software architecture in form of a textual description defined by RUP templates. Group UML was instructed to update and deliver their low-level software design in form of UML models. Our results show that iterative documentation practices led to more extensive and more detailed textual documentation. We found that writing documentation was perceived as a intrusive task leading to task specialization and allocation of documentation to less qualified team members. Consequently, this hampered collaboration within the team. Based in our findings, we suggest that if documentation is to be delivered with the project, producing documentation should be communicated and accepted by the team as a proper product. Furthermore, we argue that codification of internal development knowledge should be a non-intrusive task.

Keywords—agile teams, software development, knowledge sharing, project management, organizational management and coordination, process improvement

I. INTRODUCTION

When compared to document driven traditional software development, agile software development practices advocate an emphasis of direct communication [1]–[3]. This focus is embedded in reoccurring practices such as iterative development, frequent customer involvement, daily stand-up meetings or team-based effort estimation [4]. During the course of a development project such routines make work more predictable, support the transfer of knowledge and ensure communication within the team. While knowledge sharing practices based on socialization of team members enable agility and common understanding at the team level [5] they can also result in problems such as gaps of undocumented knowledge and lacking architectural integrity between projects. This is particularly problematic if people

leave the team or organization [1], [6], [7]. In this sense, agile development may make project management and program level management more difficult.

Efficient teamwork in agile development relies fundamentally on shared mental models [8]. Artifacts in agile development lose much of their effectiveness if participants lack a shared, overall understanding of project objectives. Sharp et al. [9] point out that the artifacts largely lack detailed information about the application under development. Non-functional requirements, for example, are difficult to link to user stories and tend to be ignored or ill-defined [6]. Code comments are generally accessible to technical staff only. Contrarily, in highly regulated and contract-driven environments formal documentation is a must. In European research projects, for example, project partners are encouraged to use documentation deliverables to disseminate their knowledge and to stimulate collaboration among the participating projects.

Software practitioners tend to perceive writing documentation as a burdensome side-task and literature suggests that this results in deliverables usually compiled and submitted at the end of a project [3]. Software projects are executed under strict constraints of time and budget. Team members are often needed for new projects and committed to these projects early on. There is rarely enough time to document all experiences and knowledge after the project delivery. Small teams might perceive codification of their knowledge less important, according to earlier findings [7] and according to our own experience this can lead to gaps in knowledge. As knowledge is not transferred seamlessly these gaps can hinder the agility of an organization as a whole.

To improve our understanding of knowledge codification strategies in software development we designed and executed a quasi-experiment in which students were required to iteratively develop a small-to-medium size application in teams. The 28 students were divided into 8 teams and two groups: SAD and UML. Group SAD (Teams A-D) was to update and deliver their high-level software architecture in form of a textual description defined by RUP templates.

Group UML (Teams E-H) was instructed to update and deliver their low-level software design in form of UML models. In this paper, we report on this study into the team’s practices to keep documentation up-to-date.

II. RELATED WORK

In their seminal theory of knowledge creation, Nonaka and Takeuchi [10] explain knowledge creation as resting in conversions between explicit and tacit forms - and between the ontological levels of individuals and groups. Nonaka and Takeuchi identify four modes of conversion in which new knowledge emerges; socialization, externalization, combination and internalization. A key point in their argument is that innovation (a result of knowledge creation) must be enabled by encouraging and facilitating the occurrence of all conversion modes in daily practices within the organization. Because software development is fundamentally driven by the creation of new knowledge we believe that Nonaka and Takeuchi’s model is a relevant conceptual model to gain understanding of knowledge processes in agile methods.

Agile methods derive much of their agility by giving preference to social interaction among people rather than documentation. The socialization mode of Nonaka and Takeuchi is therefore more prominent. Agile methods encourage knowledge creation through practices of frequent interaction among the the team members [5]. Consequently, however, agile methods may demand a more ‘social attitude’ among team members compared to plan-driven methods [11]. Furthermore, socialization incurs inherent scalability limitations. Socialization works well for small teams or teams that can be synchronized via frequent meetings [12] but for teams whose members are spatially separated or knowledge must be passed on beyond the team members, the organization is often forced to rely on explicit forms of knowledge (see Figure 1). In this paper we refer to the knowledge recipient at the program level - in contrast to the project level, which refers to the project team.

Few empirical studies have examined practices of inter-project knowledge sharing and externalization in agile development, and most work is focused on requirements engineering [6] and software process improvement [13]. Laanti [12] proposes that knowledge sharing in Scrum can be up-scaled to the program level by means of two nested control loops consisting of a team level backlog and a program level backlog. This way, next to the common Scrum daily and sprint meetings, the program level would include a *program Scrum* once or twice a week to synchronize the program level with the team level. Postmortem reviews have been successfully applied in agile projects to externalize and transfer project experiences enabling software process improvement [13]. Sharp et al. [9] point out that agile artifacts such as user stories and project wall work well supporting the project process but largely lack detailed information about the application under development.

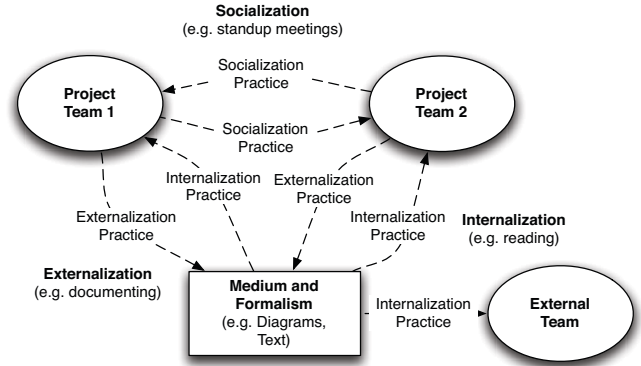


Figure 1: Problem domain: Model of knowledge transfer through agile practices

Current studies on the use of documentation during software development are few and present mixed results, studies on documentation in agile settings are even fewer [2]. Stettina and Heijstek [7] found that agile practitioners in their data set perceive documentation important or even very important but that too little documentation is available in their projects, and indicate that knowledge is lost for older projects. Clear [3] points at the behavior of students and observes documentation being seen as a burdening by-product and being hurriedly pieced together at project end. Past studies suggest that documentation is frequently out of date and should be rather seen as a communication medium than an accurate or up-to-date artifact [14].

Based in the present state of the art we find it appropriate to explore in more depth how documentation work is carried out in agile teams, and in particular the relationship between documentation formalism and developer practice. Our guiding hypothesis is that documentation formalism influences agile practices (compare Figure 1). We thus pose the following two research questions:

- 1) *How do externalization formalisms influence documentation practices in agile teams?*
- 2) *What are the implications of iteratively updated internal documentation on the quality of artifacts, perceived amount of work and project satisfaction?*

III. METHOD

We applied a mixed methods approach to collect data [15]. Qualitative process descriptions were compared and contrasted with quantitative questionnaires and the artifacts developed in course of the project. Encouraged by agile methods’ emphasis on self-managing teams and the integration of individuals into the software development process [16], we sampled individual team members’ perceptions using anonymized questionnaires. To create a deeper picture supporting our research we further used informal interviews and conducted a document inspection on the created artifacts.

Collecting practices of writing documentation as human behavior requires both qualitative data and observations in a real world context. Following Langley’s [17] framework for building theory from process data we have selected visual mapping which requires at least five cases in moderate level of detail to begin a pattern identification. Using graphical forms allows the presentation of large amount of information in little space and is a useful tool to develop and verify ideas in theory development [17].

A. Research Design

To address the research questions we embedded the study within a software engineering project course at Leiden University with a class of 28 students divided into 8 groups. This provides a suitable environment allowing a comparison of several teams working on the same project.

The project was designed as a fourteen week quasi-experiment including seven development iterations with the aim to develop a program being able to recognize and extract UML class diagrams stored in bitmap images. Guided by Richards [18] we devised project-based courses that encouraged students to engage in practical work, form their own groups and stimulated students’ individual reflections on ongoing accomplishments. While we had given recommendations, the students were given free choice of development platform and libraries. The coaching took place after the weekly software engineering lectures in nine sessions. In each session we discussed the progress of the teams, the challenges and the next steps. For instruction we used handouts in class explaining the course of the project and actions to be taken.

Students were instructed to document their development efforts focusing on architectural aspects [19]. As we wanted to study the implications of documentation formalisms we have chosen two of the most common artifacts to be updated: (1) high-level software architecture in form of a textual description and (2) low-level software design in form of UML models. We explicitly did not incorporate requirement descriptions in our study as those are already addressed by agile artifacts [9].

Due to the lectures being originally aligned to the Rational Unified Process, we decided to use RUP templates for the textual deliverables: Software Development Plan, Software Requirements Specification, Configuration Management Plan and the Software Architecture Document. One reason to choose RUP templates was that in many established software companies such are seen as standard and are expected to be in use even while applying agile methods. We emphasized throughout the project that the formal templates should be seen as guidelines rather than as stiff documents to be filled. The UML models to be delivered consisted of a class diagram, a sequence diagram, an activity diagram and a state chart diagram.

Table I: Project course

Project Planning and Initial Design
08-09-2011: (Session 1) Introduction
15-09-2011: (Session 2) Requirements Elicitation Session
22-09-2011: (Session 3) Requirements Elicitation Session 2
29-09-2011: (Session 4) Requirements Review Presentation
06-10-2011: (Session 5) Architecture and Design Presentation
Development
13-10-2011: Sprint 1 - User Interface
20-10-2011: Sprint 2 - Integrate shape recognition and OCR libraries
27-10-2011: Sprint 3 - Recognize outer UML shapes (Session 6)
03-11-2011: Sprint 4 - Integrate shape and character recognition
10-11-2011: Sprint 5 - Recognize relationships
17-11-2011: Sprint 6 - Optimization
24-11-2011: Sprint 7 - Additional Sprint (Session 7)
Delivery and Postmortem
01-12-2011: (Session 8) System Demonstration and Handover
07-12-2011: (Session 9) Project Closing: Discussion & Postmortem Review

After the definition of an initial architecture in session five the student teams were divided into two groups: Teams A-D were required to update and deliver the low-level software design models with every sprint. Teams E-H were required to update and deliver the software architecture document with every sprint. From now on we will refer to those teams as the UML and the SAD groups. The students were graded in the following schema: 25% Requirements Engineering, 25% Software Design, 25% Implementation and 25% overall quality. The updated artifacts have been made an explicit “customer requirement” the students would be graded for.

In order to test how the students perceive the importance of documentation artifacts after project delivery we chose a setup similar to the one used by Smith and Mann [20] where at the end of the development phase the student groups would swap their project source code repositories and documentation to simulate a maintenance environment. The project is hosted at Google Code and is freely accessible¹.

B. Qualitative Data Sources: Coaching Sessions, Informal Interviews & Documentation Artifacts

To address the research question we first have to clarify our research lens on human action. As patterns of action we understand those as defined by Cohen et al [21] as (i) Recurring (they can take place at different times, or involve different actors), (ii) selectable (meaning that there are forces that make them more or less likely to happen) and (iii) set in an organizational context (the actions are not those of an isolated individual).

In order to collect patterns of action after system delivery we asked the student groups to discuss and draw the process of documenting their software as UML activity diagrams on a paper and to present them one after another at the board during the closing meeting (Session 9). We chose to use activity diagrams as we assumed those to be most accessible

¹<http://code.google.com/p/image-to-uml/>

to software engineers. At the end of the session we took pictures of the models on the board and collected those drawn on paper earlier. We also kept an audio recording of the session for reference. We conducted direct observations only during the sessions in class. However, as the artifacts were mainly produced outside the classroom we conducted informal interviews and asked the students for a better level of detail or for clarification regarding certain choices or team member roles (*"How did your team assign the work related to documentation?"*, *"Why did you choose to work on documentation at that particular stage?"*). During the final session we also conducted a postmortem review [13] of the project to identify good and bad experiences of the development teams. This final session lasted for about three hours. At the end of the project the created documents were collected from the repository and assessed according to their size in word count and quality. We analyzed the content of the deliverables for each group according to the structure of the templates and readability.

C. Quantitative Data Sources: Questionnaires & Source Code Repository

We had access to the following quantitative data sources: (1) the longitudinal survey answers collected each week in class, (2) longer questionnaires conducted in each development phase and the (3) online repository. First, a short questionnaire for a longitudinal collection of the individual team member's satisfaction with the project, the amount of work and the documentation was used. This part of the questionnaire was collected every week during the project. The objective was to measure how team member perception on work environment and documentation changes during course of the challenging project, similarly to the project satisfaction graph as earlier presented by Moe [8]. The questions were: *"How satisfied are you with the project?"*, *"How satisfied are you with the amount of work?"*, *"How satisfied are you with the teamwork in your team?"* and *"How do you feel about documentation in your project?"*. Second, longer questionnaires were applied in the course of the development phase, at project delivery and after project handover to maintenance. The longer questionnaires were used to collect perceptions on the usefulness of specific artifacts, the distribution of roles within the groups and gave students the opportunity to anonymously comment on their projects. Sample questions were: *"What was your predominant role in the project?"*, *"How useful did you find documentation for your project?"*, *"How useful did you find updating and delivering documentation with every milestone?"*, *"What influence did the requirement of updating documentation have on your project satisfaction?"* and *"If you would know that after development the project will be maintained by another team, which documentation artifacts would you choose to keep up-to-date?"*. Lastly, the online repository was used to track the deliverables

and their contributors. In the repository we could track the distribution of roles within the teams, when deliverables have been checked in and by whom.

IV. RESULTS

In this section we will proceed to address the two research questions. The project started on September 8, 2011 with an introductory session and proceeded as outlined in Table I.

A. Patterns of Team Practice

Figure 3 visualizes each team's practice to create and update their internal documentation as emerged from observations, the final session in class, the informal interviews and the online repository. Actions marked dark gray were assigned to a single member of the team throughout the whole project, while actions marked light gray were rotated.

The project was somewhat challenging in terms of programming skills for the second year students as some of them expressed (e.g. *"Teamwork would be so much better if everyone was a skillful programmer..."*) and the majority of teams established specialization among their team members. While the initial version of the artifacts has been developed commonly, five of the eight teams (teams A,D,F,G, and H) defined a single team member to do the updates. because *"it was most convenient"* i.e. because that particular member was least good in coding or his own preference to work on documentation. As we can see two teams established a review process. In three of the teams the appointed member used SVN logs to update documentation. An exception here were team B and E as, we can recognize in Figure 3. In team B every developer was in charge of bringing the documentation up-to-date with his changes in code. The team had a skillful programmer leading the project and employed pair-programming. Team E kept track of hours spent on the project (keeping track of hours was a recommendation given to the students during the sessions), the person who had the least effort spent before end of the iteration had to update the documents. For team C updating documentation was not really an issue. After creating the initial documents the team was struggling to get the implementation to work, thus changes were not necessary. As emerged from the survey, figure 2 represents the team member roles of the participants as specified. Multiple answers were possible and out of the three categories, "Code", "Documentation" and "Administration" the majority of the participants (63%) specified their predominant role as "Code" only, 11% as Documentation and 0% specified their role as purely administrative. When clustering the categories in figure 2 we can see that 78% of the participants were involved in coding (coding, coding & documentation and administration & coding) and 30% were involved in documentation. Interestingly, as we can see in Figure 2 is that only 8% of the participants defined to be involved in coding and documentation.

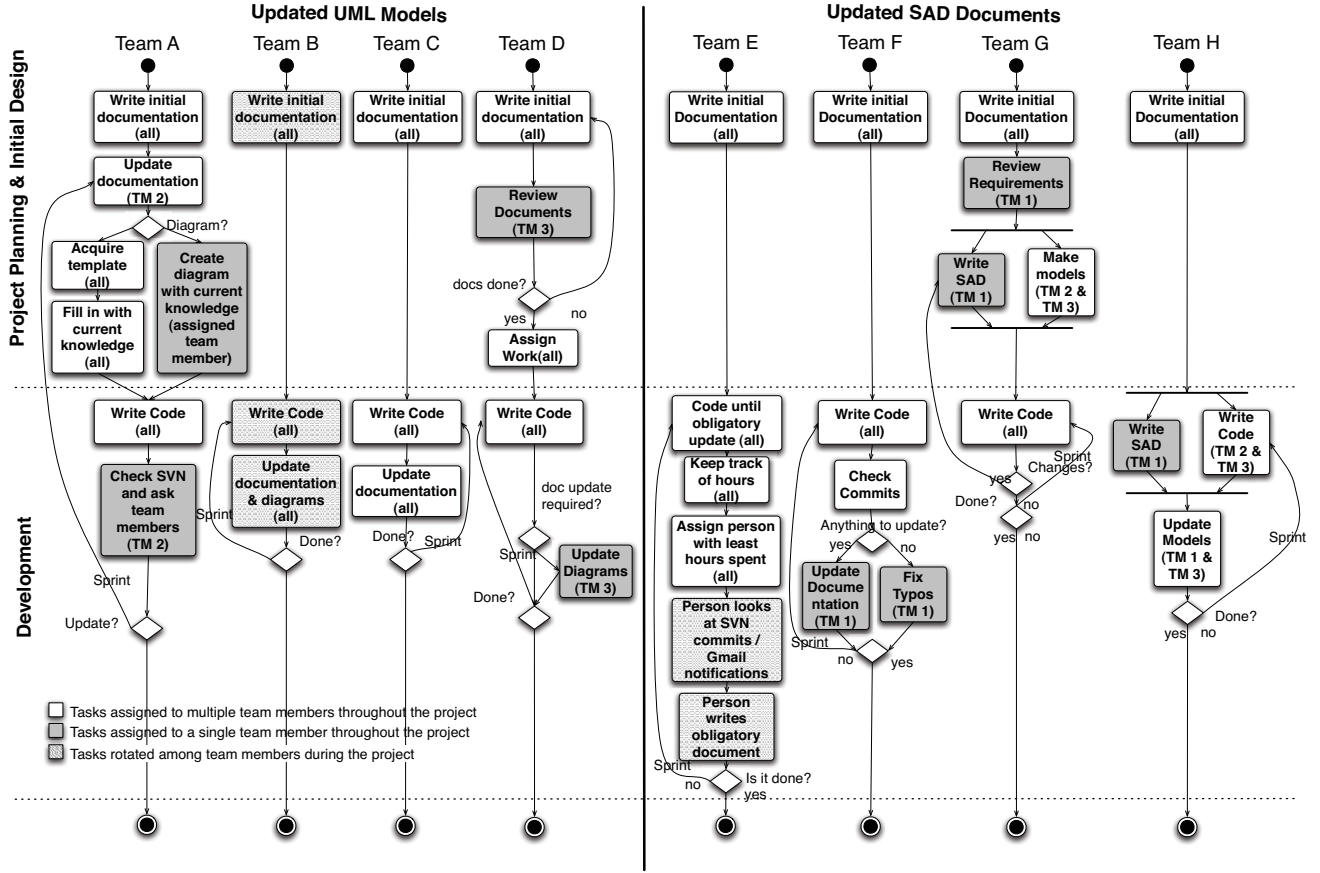


Figure 3: Team practices to update documentation

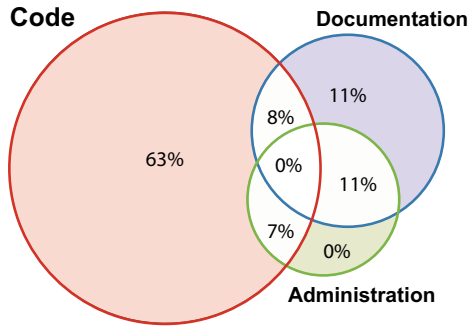


Figure 2: Distribution of team roles. Total number of participants involved in coding: 78%, in documenting: 30% and in administration: 18%

B. Project and Teamwork Satisfaction

The students generally welcomed the project while finding it challenging with comments varying from *"It will be tough getting everything to work on the deadline"* to *"Very nice project. Challenging, yet very interesting."*. During the re-

Table II: Summary of team practices to write and maintain documentation

Team A	All members worked on initial versions, thereafter one team member was in charge of maintaining documents, "according to repository and team member comments"
Team B	Every developer was in charge of updating documentation with changes in code. Difficult parts of the program were developed in pair programming.
Team C	Updating models was not really an issue. Most of the architecture was done in UML models, and as the team was struggling to implement the initial design there was not really much to change.
Team D	All team members but one wrote the initial documents, the remaining team member reviewed them now and then.
Team E	Kept track of hours, the person "who had time" updated the artifacts. After asking the team how many were involved, a team member answered "basically all".
Team F	Initial documents were created by all team members, updates were done by team member TM1.
Team G	While one team member was in charge of updating the textual SAD documentation two others were in charge of UML models
Team H	Initial by all, updates were committed by a single team member because he was the "least good in coding"

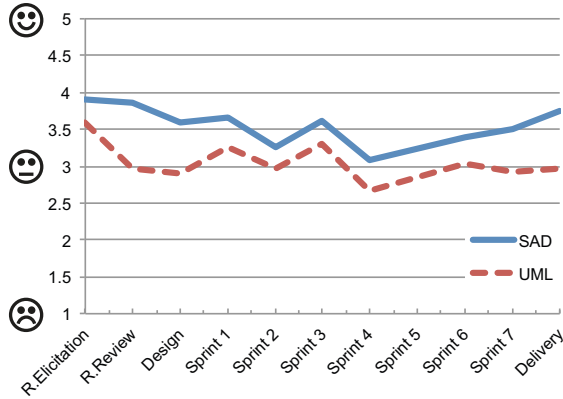


Figure 4: Project satisfaction. [Scale: 1=Not at all satisfied, 2=Slightly satisfied, 3=Moderately satisfied, 4=Very satisfied, 5=Extremely satisfied]

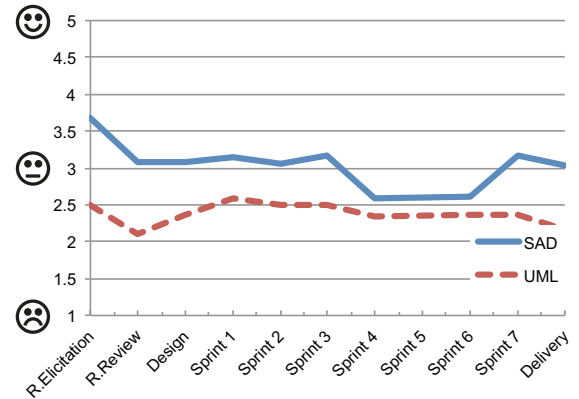


Figure 5: Satisfaction with the amount of work. [Scale: 1=Not at all satisfied, 2=Slightly satisfied, 3=Moderately satisfied, 4=Very satisfied, 5=Extremely satisfied]

quirements review session there was quite some uncertainty due to the free choice of implementation platform. During the implementation phase the project remained challenging, and at the end of the first iteration a student commented: *"Using the internet, we have made good progress. However, because the code is from mixed sources and adapted to our situation, the code is a mess. The documentation isn't up to date either, though I do think we are all on the same level."* During the feedback session one student remarks that after integrating the OCR and shape recognition libraries many discovered how much work it really was.

Figures 4 and 5 illustrate the satisfaction with the project and the amount of work for both customer groups during the course of the project. In Figure 4 we can observe that the curves for satisfaction with the project run parallel, with the UML groups being generally less satisfied with. Figure 5 illustrates the satisfaction with the amount of work for the SAD and UML groups. The teams perceived updating and delivering documentation as not very useful. When asked on how much influence documentation work would have on their project satisfaction, they said it would have only slight influence (compare Fig. 7).

Teams perceived documentation as a burden, as work 'that needs to be done'. Especially for the SAD group we can observe an increase in perceived amount of documentation in the period between Sprint 3 and 7 of the development phase. This increase of perceived documentation amount as visualized in Figure 6 happens at the same time as the team members perceive an increased amount of work (compare Figure 5). Interesting is to note that when asked which documents the teams would keep up to date if the project is to be handed over, majority of the participants chose software architecture document, software design models and software requirements specification out of the five deliverables.

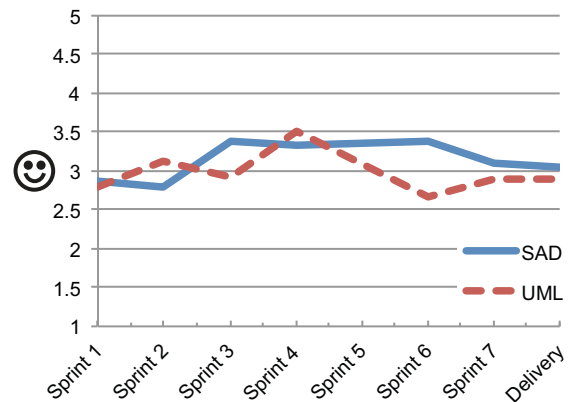


Figure 6: Perceptions on internal documentation in course of the implementation phase. [Scale: 1=Way too little, 2=Slightly too little, 3=Just about right, 4=Slightly too much, 5=Way too much]

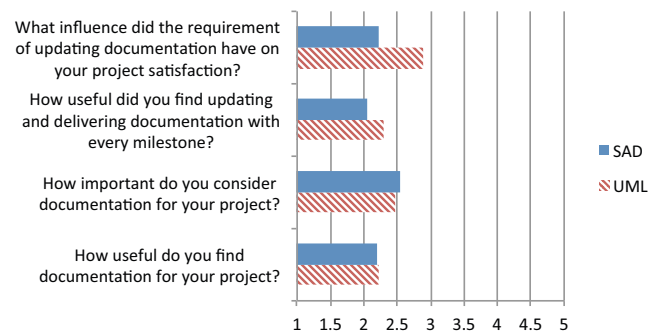


Figure 7: Perceptions after project delivery. [Scale: 1=Not at all, 2=Slightly, 3=Moderately, 4=Very, 5=Extremely]

C. Quality of Documentation

After completion of the project on December 7 we downloaded the latest versions of the deliverables and conducted a document analysis. Table III illustrates the results of our analysis. As we can see in the table the teams instructed to update their architecture document with every milestone generally produced larger and more elaborate documents. The quality of the artifacts delivered by the SAD groups was significantly better. Teams that were instructed to update their UML models did so, however as one could expect, the changes within the models were less visible. As they did not had to update other artifacts than the models, the quality of the documents delivered was lower significantly, for example the documents contained template explanations still. Table III also contains the assessment of the implementation, which indicates that there is no significant correlation of the updated documentation on the quality of the implementation. The software development plan was largely omitted by the teams which made it the documentation artifacts with the lowest quality delivered.

V. DISCUSSION

In this section, we discuss the emergent patterns of action, the team members' expressed satisfaction and the quality of documentation deliverables. We will first consider the role of documentation as 'product', e.g. as tangible project deliverables. Second, we will consider the role of documentation as a medium of knowledge exchange and how documentation supports sharing and leveraging project knowledge at the organization's program level.

A. Documentation as a Product: Iterations Produce Better Textual Documentation

The qualitative comparison of the deliverables in Table III illustrates that textual documentation benefits from being build up iteratively. The benefits of iterative practices on diagrammatic documentation are less visible in our data. While the updates of UML models, as a more formal medium, were less visible, the updates of the teams assigned to update the less formal textual architecture documents produced more extensive artifacts.

The visualization of actual patterns of actions within the different teams (see Figure 3) shows how feedback resulting from multiple forms of knowledge conversion [10] is embedded into the documentation. Hence, emergence of textual documentation artifacts can be clearly linked to the iterative development process. This may help to improve transparency towards the quality management of documentation deliverables (or 'documentation products').

However, one has to consider another aspect: In our designed experiment, team members did not perceive documentation as contributing value to their projects. Team members referred to it as *"a task that needs to be done."* Motivation to participate in an activity increases with the

perception of importance and influence of the activity's results [22]. Therefore, we find it reasonable to suggest that writing documentation becomes more attractive if someone, and preferably an acknowledged stakeholder, has expressed an interest in the results. A comment of a Dutch Scrum coach in a recently conducted interview pinpoints the issue: *"I always try to explain, "You define what amount of documentation you want, I am happy to provide you with all kinds of documentation, but I need someone who is happy with what I deliver"."* Clients should acknowledge for the team that documentation is work of equal importance as other tasks, and their requirements towards documentation should be as clear as other requirements. In short, documentation should be considered a valuable product.

B. Documentation as a Medium: Creating Artifacts While Gambling With Collaboration

SAD teams expressed an increase in the amount of work on documentation tasks lasting throughout the development phase. This corresponds with their perception of an increasing amount of documentation being available. Among the UML teams the perception of the amount of work stayed more constant during the project and the UML teams perceived only a short peak in the amount of documentation.

One can argue that the SAD teams perceived a surplus of available knowledge during development as the team members work tightly together. This would be consistent with our earlier findings where a team member commented: *"During project development any needed documentation is generally available. However, finding documentation for older projects is not always easy, and sometimes this documentation is missing."* [7]. However, one can also interpret this increase as a distraction that the teams perceived from the documentation task. This interpretation is supported by the perception of increasing amount of work. It suggests that updating UML models was considered less intrusive, and less demanding than updating textual documentation.

Our data shows that the majority of teams implemented strict roles dividing coding and documentation work. Most teams appointed documentation work to a single team member, most frequently the least qualified programmer. While all in all 78% of the participants were involved in coding, 30% worked on documentation and only 8% answered to have worked on both (see Figure 2). This is very far from the 'agile ideal' where team members are generalists and avoid specialized roles. The circles would then have much greater overlap. Agile team members do not have specific roles, instead each team member has a variety of roles. This encourages socialization within the team and is an important enabler for knowledge creation [10].

An interesting aspect we discovered in the data was a strong tendency towards increasing role enforcement throughout the duration of the project. In the initial phases all team members were involved in writing documentation but

Table III: Descriptive variables and team results (**lowest** & **highest** values)

	<i>project team</i>	Updated UML Models				Updated SAD Documents			
		A	B	C	D	E	F	G	H
	<i>persons in team</i>	4	4	3	4	4	3	4	3
	<i>environment</i>	C++	C++/PHP	C++	C++	C++	C#	C++	C#
	<i>implem. quality</i>	● ○ ○ ○	● ● ● ●	● ● ○ ○	● ● ○ ○	● ○ ○ ○	● ● ● ●	● ● ○ ○	● ● ● ●
<i>wordcount</i>	<i>SAD</i>	1051	670	508	670	1601	1036	1123	2296
	<i>SRS</i>	1238	850	1347	326	1116	2167	1757	2393
	<i>CMP</i>	757	377	700	578	1143	717	-	1747
	<i>SDP</i>	-	1692	648	602	-	912	-	-
<i>quality</i>	<i>SAD</i>	● ○ ○ ○	● ○ ○ ○	● ○ ○ ○	● ○ ○ ○	● ● ○ ○	● ○ ○ ○	● ● ○ ○	● ● ● ●
	<i>SRS</i>	● ● ○ ○	● ○ ○ ○	● ● ○ ○	● ○ ○ ○	● ○ ○ ○	● ● ● ●	● ● ● ●	● ● ● ●
	<i>CMP</i>	● ○ ○ ○	● ○ ○ ○	● ○ ○ ○	● ○ ○ ○	● ● ● ●	● ● ○ ○	-	● ● ● ●
	<i>SDP</i>	-	● ● ○ ○	-	● ○ ○ ○	-	● ○ ○ ○	-	-
<i>class diagram coupling</i>		low	mid-high	low	low	mid-low	mid-high	mid-low	mid-high
<i>activity diagram complexity</i>		mid	low	mid	mid	high	mid	mid	high
<i>documentation participation</i>		single	all, on changes	single	single	all, on effort	single	single	single

gradually the ‘documentation role’ became the province of a single team member. We speculate that time pressure is an important explanation for this tendency. Time pressure has been found to have an impact on the choice and maintenance of team routines. Even if an inadequacy has been indicated beforehand, time pressure increases the likelihood to choose an established routine [23]. Although perceived as highly important, time pressure and cognitive demands of documentation tasks impede sharing of documentation work. Explicit rotation of roles and more effective documentation tools may create the necessary stimulus for sharing the documentation tasks [24].

The analysis of the team practices in Fig. 3 and the distribution of roles in Fig. 2 illustrates that the joint effort of producing documentation faces other obstacles than the availability of documentation templates and stated documentation requirements. As we can see in Fig. 7, both groups similarly perceive the usefulness and importance of internal documentation. As one team member comments: “Documentation was written because it was required, but because of the program’s modularity, no documentation other than the original class diagram (and of course OpenCV, Tesseract [library] reference manuals) was necessary contributing to the project.”. While looking at the teams’ documentation practices in Figure 3 one can recognize how the developed patterns of action, also called organizational routines [23], enable the feedback of team members and source code to be embedded in documentation. However, the circular knowledge creation processes are missing [10]. This suggests that increasing insights - stemming from documentation work - was not incorporated back into the source code.

Hence, we can see that the unbalance in documentation practices distracts from coding activities while not sufficiently adding value to the project. This led the team to

perceive documentation as an intrusive task. As opposed to the incremental nature of development from agile practice, neither the formal documentation templates nor the UML models alone were very suited to support collaboration within the project teams. These observations contradict the studies of Sharp et al. [9] which demonstrate how the notational attributes and social processes of agile artifacts such as user stories and the project wall applied in practice are mutually supportive for an agile team. For better externalization of development knowledge these reoccurring patterns of knowledge creation activity should be more balanced.

C. Agile Program and Portfolio Management: Usability and Importance of Knowledge Sharing

“There is one thing we did not implement, we did not use RUP”, said team member of team B jokingly at the beginning of the system demonstration while summarizing which requirements have been met (Using RUP was not explicitly a requirement, but was widely covered in the lectures). In the short maintenance phase of the project (see project plan in Table I) the teams reported not to have taken any substantial advantage of the produced documentation artifacts other than a short handover description that was to be prepared. However, team members do perceive that documentation is important for future development. When asked which documents the teams would keep up to date when the project was to be handed over, out of the five artifacts vast majority of the participants chose for software architecture document (75%), software design models (71%) and software requirements specification (54%) to be kept up-to-date. This is also visible through the difference on perceived usefulness and importance as depicted in the two questions in Fig. 7. While the team members perceive it less useful within their teams now, they perceive that

documentation is more important, thus indicating a tension between team and external stakeholder's interests.

As we can see in Fig. 7, the teams perceived updating UML models more useful and perceived it to have a bigger influence on their project satisfaction than updating SAD documents. In organizational science Carlile [25] discussed documentation artifacts as types of boundary objects helping to bridge functional and organizational boundaries. Participants involved in cross-boundary knowledge exchange need to be aware of the personal costs and the necessity to transform own as well as influence knowledge in other domains. Effective boundary objects stimulating collaboration are a prerequisite to that. Taking our findings one can argue that the UML models are more effective boundary objects according to Carlile [25] due to their visual representation and less effort necessary for alternation. This, however, is only true if they are created in a format easily adaptable to all participants. Software and file formats available to a single functional group only can hinder feedback of participants from other functional groups.

To conclude, for management of agile project programs and portfolios it is important to note that since team members perceive inter-project knowledge transfer as important, it is just that they oppose how the applied codification practices were implemented in their particular projects.

VI. CONCLUSIONS

In this paper, we presented an in-depth analysis of documentation practices in small software teams. Our combination of quantitative and qualitative data analysis allowed us to shed light on longitudinal and process-centric aspects of the documentation work carried out in the teams - covering a period of 14 weeks.

Dividing our analysis into the perspectives of *documentation as a product* and *documentation as a medium*, we found that primarily textual artifacts benefited from iterative documentation practice. Iterative practices thus not only help improving source code but can also improve the quality of documentation deliverables. We found that teams perceive documentation created for internal knowledge sharing as a distraction which causes them to choose the least qualified programmer to update documentation. This perception of documentation as what we can call an *intrusive task* is a reason why developers in small teams don't like to write documentation - it distracts them from doing what they find most important: writing code. When developed for internal purposes one can conclude that the formal templates are able to capture the development knowledge. However, they seem not well suited to support collaboration within the team members as the majority of the teams chose a single team member to update the documents. This specialization is hampering collaboration in self-managing teams.

With respect to management practices for agile project programs and portfolios we conclude that the formal docu-

mentation was perceived as a burden by the teams as it was not contributing to the development of the software, however, knowledge sharing for future projects and maintenance was still perceived as important. Team members noted that if the project was to be handed over that they would keep software architecture document, design models and requirements specification up-to-date. This implies that the teams perceive knowledge sharing as important, however, they are unsatisfied how design knowledge has been documented.

A. Recommendations for Research and Practice

In this paper we have employed a new longitudinal approach to measure team and task satisfaction, based upon the project satisfaction graph as applied by Moe [8]. The notation can help measuring the implications of events occurring during development on team satisfaction. Such events can be the dependency of team satisfaction on external factors. The two project satisfaction graphs (Fig. 4) running similarly indicate that both groups perceive a similar satisfaction in course of the project while the "humps" indicate the uncertainties encountered. We further discuss documentation as an intrusive task during development, we also present an approach how to quantify it, but the question to be solved yet is what non-intrusive documentation should look like.

We recommend that for *documentation as a product* (e.g. in regulated and contract-driven environments) it is recommended to develop and deliver textual documentation iteratively by the use of easily accessible and adjustable artifacts as this will most likely improve the quality of deliverables and contribute to promote collaboration between people in different functional domains. In any case, however, documentation needs to be communicated and accepted by the team as a proper product, product owners should motivate it as such and should be clear on their requirements. Furthermore, we argue that *documentation as a medium* (e.g. for sharing and leveraging of internal development knowledge) should be a non-intrusive task.

B. Validity Considerations

Although industrial teams are expected to have more experience than the chosen software engineering students, we argue that the challenging project environment with strict time pressure is very likely to provide an environment with similar implications on perceptions on documentation and project satisfaction. The extent to which graduate, undergraduate and doctoral students are representative of professional software developers and the threat of interaction of selection and treatment respectively, have been addressed in various other studies (e.g. [26], [27]).

Considering the ease of use and the findings that scales with two, three, or four response categories yield least reliable scores [28], we have decided on using a 5-point, unipolar scale. To reduce bias we encouraged the respondents to provide their honest opinions by emphasizing the

anonymous treatment of data. We re-iterated that the data collection had no implication on grading.

ACKNOWLEDGMENT

This work has been supported by the Living Lab The Hague project funded by the European Commission, the Municipality of The Hague and Leiden University. We thank all the participants for generously contributing to this study.

REFERENCES

- [1] P. Abrahamsson, J. Warsta, M. T. Siponen, and J. Ronkainen, "New directions on agile methods: a comparative analysis," in *Proceedings of the 25th International Conference on Software Engineering*, ser. ICSE 2003. Washington, DC, USA: IEEE Computer Society, 2003, pp. 244–254.
- [2] T. Dybå and T. Dingsøyr, "Empirical studies of agile software development: A systematic review," *Information Software Technology*, vol. 50, no. 9–10, pp. 833–859, 2008.
- [3] T. Clear, "Documentation and agile methods: striking a balance," *SIGCSE Bulletin*, vol. 35, no. 2, pp. 12–13, 2003.
- [4] T. E. Fægri, "Adoption of team estimation in a specialist organizational environment," in *In proceedings of XP2010, 11th International Conference, Trondheim, Norway*, ser. LNBIP, vol. 48, 2010.
- [5] G. Melnik and F. Maurer, "Direct verbal communication as a catalyst of agile knowledge sharing," in *Proceedings of the Agile Development Conference*. Washington, DC, USA: IEEE Computer Society, 2004, pp. 21–31.
- [6] B. Ramesh, L. Cao, and R. Baskerville, "Agile requirements engineering practices and challenges: an empirical study," *Information Systems Journal*, vol. 20, no. 5, pp. 449–480, 2007.
- [7] C. J. Stettina and W. Heijstek, "Necessary and neglected? An empirical study of internal documentation in agile software development teams," in *Proceedings of the 29th ACM international conference on Design of communication*, ser. SIGDOC '11. New York, NY, USA: ACM, 2011, pp. 159–166.
- [8] N. B. Moe, T. Dingsøyr, and T. Dybå, "A teamwork model for understanding an agile team: A case study of a scrum project," *Inf. Softw. Technol.*, vol. 52, pp. 480–491, May 2010.
- [9] H. Sharp, H. Robinson, and M. Petre, "The role of physical artefacts in agile software development: Two complementary perspectives," *Interacting with Computers*, vol. 21, pp. 108–116, January 2009.
- [10] I. Nonaka and H. Takeuchi, *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press, USA, May 1995.
- [11] S. Nerur, R. Mahapatra, and G. Mangalaraj, "Challenges of migrating to agile methodologies," *Commun. ACM*, vol. 48, pp. 72–78, May 2005.
- [12] M. Laanti, "Implementing program model with agile principles in a large software development organization," in *Proceedings of the International Computer Software and Applications Conference*, ser. COMPSAC '08. Washington, DC, USA: IEEE CS, 2008, pp. 1383–1391.
- [13] T. Dingsøyr and G. K. Hanssen, "Extending agile methods: Postmortem reviews as extended feedback," in *4th International Workshop on Learning Software Organizations*, 2002, pp. 4–12.
- [14] A. Forward and T. Lethbridge, "The relevance of software documentation, tools and technologies: a survey," in *ACM Symposium on Document Engineering*, 2002, pp. 26–33.
- [15] M. Miles and A. Huberman, *Qualitative Data Analysis : An Expanded Sourcebook*, 2nd ed. Thousand Oaks: Sage, 1994.
- [16] C. J. Stettina and W. Heijstek, "Five agile factors: Helping self-management to self-reflect," in *Proceedings of European Software Process Improvement Conference (EuroSPI 2011)*, Roskilde, Denmark, 2011.
- [17] A. Langley, "Strategies for Theorizing from Process Data," *The Academy of Management Review*, vol. 24, no. 4, pp. 691–710, Oct. 1999.
- [18] D. Richards, "Designing project-based courses with a focus on group formation and assessment," *Trans. Comput. Educ.*, vol. 9, pp. 2:1–2:40, March 2009.
- [19] T. C. Lethbridge, "What knowledge is important to a software professional?" *Computer*, vol. 33, pp. 44–50, 2000.
- [20] L. Smith, S. Mann, and N. Buissink-Smith, "Crashing a bus full of empowered software engineering students," *New Zealand Journal of Applied Computing and Information Technology*, vol. 5, pp. 69–74, 2001.
- [21] M. D. Cohen, R. Burkhart, G. Dosi, M. Egidi, L. Marengo, M. Warglien, and S. Winter, "Routines and Other Recurring Action Patterns of Organizations: Contemporary Research Issues," *Ind Corp Change*, vol. 5, no. 3, pp. 653–698, Jan. 1996.
- [22] J. G. March and H. A. Simon, *Organizations (2nd ed.)*. Cambridge, Massachusetts: Blackwell Publishers, 1993.
- [23] M. C. Becker, "Organizational routines: a review of the literature," *Ind Corp Change*, vol. 13, no. 4, pp. 643–678, Aug. 2004.
- [24] T. E. Fægri, T. Dybå, and T. Dingsøyr, "Introducing knowledge redundancy practice in software development: Experiences with job rotation in support work," *Inf. Softw. Technol.*, vol. 52, pp. 1118–1132, October 2010.
- [25] P. R. Carlile, "A pragmatic view of knowledge and boundaries: Boundary objects in new product development," *Organization Science*, vol. 13, pp. 442–455, July 2002.
- [26] L. C. Briand, Y. Labiche, M. D. Penta, and H. D. Yan-Bondoc, "An experimental investigation of formality in UML-based development," *IEEE Trans. Software Eng*, vol. 31, no. 10, pp. 833–849, 2005.
- [27] M. Höst, B. Regnell, and C. Wohlin, "Using students as subjects: a comparative study of students and professionals in lead-time impact assessment," *Empirical Software Engineering*, vol. 5, no. 3, pp. 201–214, 2000.
- [28] C. Preston, "Optimal number of response categories in rating scales: reliability, validity, discriminating power, and respondent preferences," *Acta Psychologica*, vol. 104, no. 1, pp. 1–15, Mar. 2000.